

وثائق عرض نونفا لأكشري فيرنيشينجز

خاص بمشروع النونفا - كل شريحة موثقة بشكل متسق وكامل PowerPoint هذا المستند هو مرجع شامل لعرض شرائح ال

بنية المستند

كل قسم يتبع نفس الصيغة للاتساق:

- ما هو؟ - شرح واضح للمفهوم.
- أين يقع في الكود؟ - مسارات الملفات والمواقع الدقيقة.
- لماذا يهم؟ - القيمة التجارية والتقنية.
- مثال الكود - كود حقيقي يعمل.

1. شريحة العنوان

في الخلفية و Node.js لشراء الأثاث الفاخر، مبني باستخدام (Full-Stack) ما هو؟ تطبيق تجارة إلكترونية متكامل في الواجهة الأمامية React/TypeScript.

أين يقع في الكود؟

- نظرة عامة على المشروع: [README.md](#), [package.json](#)
- نقطة دخول الخلفية: [backend/index.js](#)
- تحميل المسارات: [backend/src/modules/index.routes.js](#)

لماذا يهم؟ يجهز الحضور للعرض: يوضح ما سيتم عرضه والتقنيات المستخدمة في المشروع.

2. واجهة برمجة التطبيقات (REST API)

القياسي HTTP للموارد (المنتجات، المستخدمين، الطلبات) باستخدام الفعل (Endpoints) ما هو؟ واجهة تقدم نقاط نهاية

أين يقع في الكود؟

- وحدات المسارات: [backend/src/modules//routes.js](#)
- مثال مسارات المنتجات: [backend/src/modules/product/product.routes.js](#)

بطريقة متوقعة API لماذا يهم؟ يجعل السلوك موحداً للموارد بحيث يمكن لمطوري الواجهة الأمامية والأدوات التفاعل مع ال

مثال الكود:

```
productRouter.route('/').get(cacheResponse(60*1000), getAllProduct).post(protectedRoutes, addProduct)
```

3. HTTP أفعال

(حذف) DELETE، (تحديث) PUT، (إنشاء) POST، (قراءة) GET: التي تصف الإجراء الذي تنوي الطلب تنفيذه HTTP ما هو؟ أفعال

أين يقع في الكود؟

- معرف في كل مسار في: [backend/src/modules/product/product.routes.js](#)

بديهيًا ويساعد الأدوات والكاش في التعامل بشكل صحيح API لماذا يهم؟ استخدام الفعل الصحيح يجعل ال

مثال الكود:

```
productRouter.route('/:id')
  .get(getSingleProduct)
  .put(updateProduct)
  .delete(deleteProduct)
```

4. صيغة JSON

هي صيغة البيانات المستخدمة للطلبات والردود بين الواجهة الأمامية والخلفية - قابلة للقراءة من قبل الإنسان JSON ما هو؟ ومستقلة عن اللغة.

أين يقع في الكود:

محلل الجسم والإعدادات - backend/index.js في

```
import express from 'express'
app.use(express.json())
app.use(cors())
app.use(morgan('dev'))
```

Axios إعداد عميل - frontend/src/lib/api.ts في

يجعل التواصل بين الواجهة والخلفية بسيطاً ومتسقاً وسهل الفهم من قبل الأدوات JSON لماذا يهم؟

أمثلة الكود:

في الواجهة الأمامية Axios إعداد عميل:

```
export const api = axios.create({
  baseURL: apiBaseUrl,
  timeout: 20_000,
  headers: { "Content-Type": "application/json" },
})
```

(POST /api/v1/auth/signup) طلب التسجيل:

```
{
  "name": "محمد أحمد",
  "email": "mohammad@example.com",
  "password": "كلمة السر123"
}
```

(POST /api/v1/cart) طلب إضافة إلى السلة:

```
{
  "productId": "64a1f2e5b3c7d9a0f1e2d3c4",
  "quantity": 1
}
```

رد النجاح:

```
{
  "status": "success",
  "cart": {
    "items": [{ "product": "64a1f2...", "quantity": 2 }],
    "total": 1799.98
  }
}
```

5. معمارية العميل-الخادم

يتعامل معها API ما هو؟ توضح كيف يتفاعل الطرف الأمامي (عميل) والخلفي (خادم): إجراءات المستخدم في العميل تصبح طلبات الخادم.

أين يقع في الكود:

تحميل الخادم - backend/index.js في

```
import express from 'express'
import { dbConnection } from './database/dbConnection.js'
import { bootstrap } from './src/modules/index.routes.js'
import cors from 'cors'
import morgan from 'morgan'

const app = express()
app.use(cors())
app.use(express.json())
app.use('/uploads', express.static('uploads'))
app.use(morgan('dev'))

bootstrap(app)
dbConnection()

app.listen(port, () => {
  console.log(`Backend API listening on http://localhost:${port}`)
})
```

عميل الواجهة الأمامية - frontend/src/lib/api.ts في

لماذا يهم؟ يفصل المسؤوليات (منطق الواجهة في العميل، البيانات والقواعد في الخادم). يحسن الأمان والاختبارية والقابلية للتوسع.

مثال سير العملية (إضافة إلى السلة):

طلب الواجهة الأمامية:

```
api.post('/api/v1/cart', {
  productId: '64a1f2e5b3c7d9a0f1e2d3c4',
  quantity: 1
})
```

معالج الخادم:

```
// backend/src/modules/cart/cart.controller.js
export const addToCart = async (req, res) => {
  // تحقق من المدخلات
  // حدّث قاعدة البيانات
  // أرسل رد JSON
}
```

6. API استهلاك ال

مع إرسال توكن المصادقة تلقائياً (Axios) مشترك HTTP ما هو؟ كيفية استدعاء الواجهة الأمامية للخادم عبر عميل

أين يقع في الكود؟

- المشترك Axios عميل: [frontend/src/lib/api.ts](#)
- محيط الطلب: نفس الملف

لماذا يهم؟ تجميع إعدادات الطلب والمصادقة في مكان واحد بحيث لا تكرر المكونات هذا الكود

مثال الكود:

```
api.interceptors.request.use((config) => {
  const token = typeof window !== 'undefined' &&
    window.localStorage.getItem(TOKEN_KEY)
  if (token) {
    (config.headers as Record<string, string>).token = token
  }
  return config
})
```

7. المصادقة والتحقق

مع التحقق من البريد الإلكتروني والمسارات المحمية JWT ما هو؟ مصادقة قائمة على

أين يقع في الكود:

في backend/src/modules/auth/auth.controller.js :

```
const issueAuthToken = (user) =>
  jwt.sign(
    { userId: user._id, role: user.role },
    process.env.JWT_KEY
  )
```

تعريف المسارات المحمية - backend/src/modules/auth/auth.routes.js في

برمجيات المسارات المحمية - backend/src/middleware/ في

المحمية ويدعم تدفقات استرجاع Endpoints لماذا يهم؟ يضمن أن يتمكن المستخدمون المصرح لهم فقط من الوصول إلى ال وتحقق الحسابات

مثال الكود:

```
const issueAuthToken = (user) =>
  jwt.sign(
    { userId: user._id, role: user.role },
    process.env.JWT_KEY
  )
```

8. تكامل قاعدة البيانات

لتخزين والتحقق من البيانات (المستخدمون، المنتجات، الطلبات، السلات) مع Mongoose مع MongoDB ما هو؟ استخدام التحقق على مستوى المخطط.

أين يقع في الكود:

في backend/database/dbConnection.js :

```
import mongoose from "mongoose"

export const dbConnection = () => {
  const mongoUri = process.env.MONGODB_URI || process.env.DB_STRING ||
  'mongodb://localhost:27017/halim'

  mongoose.connect(mongoUri)
    .then(() => console.log("db is connected successfully"))
    .catch((err) => console.log('db failed', err))
}
```

إلخ ، user.model.js ، product.model.js جميع نماذج البيانات مثل backend/database/models/ وفي

لماذا يهم؟ يخزن البيانات بشكل دائم، يفرض التحقق على مستوى المخطط، ويدعم الاستعلامات لجميع ميزات التطبيق

مثال الكود:

```
mongoose.connect(mongoUri)
  .then(() => console.log('تم الاتصال بقاعدة البيانات'))
  .catch(err => console.error('فشل الاتصال', err))
```

9. Swagger وثائق

API توفر توثيقاً تفاعلياً قائماً على المتصفح ولوحة اختبار للـ Swagger (OpenAPI) ما هو؟

أين يقع في الكود؟

- الإعدادات: [backend/swagger.config.js](#)
- مثبت في [backend/src/modules/index.routes.js](#) تحت /api-docs

بدون كتابة كود عميل؛ يحسن قابلية الاكتشاف Endpoints لماذا يهم؟ يمكن للمطورين والمختبرين استكشاف واختبار الـ

مثال الكود:

```
import swaggerUi from 'swagger-ui-express'

app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(specs))
```

10. رفع الملفات

multipart/form-data ما هو؟ دعم رفع ملفات الصور (صور المنتجات) من العميل إلى الخادم باستخدام

أين يقع في الكود؟

- مساعد الرفع: [backend/src/services/fileUpload/fileUpload.js](#)
- مسارات رفع المنتجات: [backend/src/modules/product/product.routes.js](#)
- خدمة الملفات الثابتة: [backend/index.js](#) (`app.use('/uploads', express.static('uploads'))`)

لماذا يهم؟ يسمح بتخزين صور المنتجات وتقديمها للمستخدمين؛ مهمة لقوائم المنتجات والعرض البصري

مثال الكود:

```
uploadFields([
  { name: 'imgCover', maxCount: 1 },
  { name: 'images', maxCount: 10 }
])
```

11. خدمة البريد الإلكتروني

(للتحقق، إعادة تعيين كلمة المرور، إلخ OTP رموز) ما هو؟ خدمة لإرسال رسائل التحقق والإخطارات

أين يقع في الكود؟

- مرسل البريد: [backend/src/services/email/sendEmail.js](#)
- النماذج: [backend/src/services/email/verificationEmailTemplate.js](#)
- يتم التفعيل من: متحكم المصادقة أثناء تدفقات التسجيل

لماذا يهم؟ يمنع الحسابات المزيفة ويحسن الأمان؛ البريد ضروري لاسترجاع كلمة المرور والتواصل مع المستخدمين

مثال الكود:

```
await transporter.sendMail({
  from: senderEmail,
  to: userEmail,
  subject: 'تحقق من حسابك',
  html: verificationEmailTemplate({ name, otp })
})
```

12. معالجة الأخطاء

ما هو؟ آلية مركزية لمعالجة وتنسيق الأخطاء للرد على الأخطاء بشكل متسق

أين يقع في الكود؟

- برمجية الخطأ العام: [backend/src/middleware/globalError.js](#)

- [backend/src/middleware/validation.js](#): مساعدو التحقق

لماذا يهم؟ يمنع تسرب الأخطاء الداخلية للعملاء، يسهل تصحيح الأخطاء في التطوير، ويحافظ على اتساق الردود في الإنتاج

مثال الكود:

```
function globalError(err, req, res, next) {
  res.status(err.statusCode || 500).json({
    error: err.message
  })
}
```

13. الكاش وتخزين البيانات المؤقت

المطلوبة بكثرة Endpoints لتحسين الأداء في الـ GET ما هو؟ تخزين مؤقت قصير المدى لردود

أين يقع في الكود؟

- برمجية: [backend/src/middleware/responseCache.js](#)
- مطابقة على: [backend/src/modules/product/product.routes.js](#)
- إبطال الكاش: [backend/src/modules/product/product.controller.js](#)

في الإنتاج Redis **لماذا يهم؟** يسرع وقت الاستجابة ويقلل استعلامات قاعدة البيانات المكررة. استخدم

مثال الكود:

```
router.get(
  '/api/v1/products',
  cacheResponse(60 * 1000),
  getAllProducts
)
```

14. تحديد معدل الطلبات

خلال فترة زمنية Endpoint ما هو؟ آلية لتحديد عدد الطلبات التي يمكن لعميل إرسالها إلى

أين يقع في الكود:

برمجية تحديد المعدل في `backend/src/middleware/rateLimit.js` :

```
export function rateLimit({ windowMs = 15 * 60 * 1000, max = 20 } = {}) {
  return (req, res, next) => {
    const key = `${getClientKey(req)}:${req.path}`;
    const now = Date.now();
    const bucket = windows.get(key) || { count: 0, resetAt: now + windowMs };

    if (now > bucket.resetAt) {
      bucket.count = 0;
      bucket.resetAt = now + windowMs;
    }

    bucket.count += 1;
```

```

    if (bucket.count > max) {
      return res.status(429).json({ message: 'Too many requests' });
    }

    next();
  };
}

```

مستخدمة في `backend/src/modules/auth/auth.routes.js` :

الحساسية من هجمات القوة الغاشمة والإساءة، يقلل الأحمال المفاجئة Endpoints لماذا يهم؟ يحمي الـ

مثال الكود:

```

authRouter.post(
  '/signin',
  rateLimit({
    windowMs: 15 * 60 * 1000,
    max: 10
  }),
  validation(signinVal),
  signin
)

```

15. شريحة الإغلاق

ما هو؟ ملخص يجمع ميزات المشروع والدروس المستفادة من بناء نظام التجارة الإلكترونية

أين يقع في الكود؟

- تحميل التطبيق: [backend/index.js](#)
- معجم التوجيه: [backend/src/modules/index.routes.js](#)

لماذا يهم؟ يعزز الواجهات الرئيسية للحضور ويشير إليهم إلى مواقع الكود لمزيد من الاستكشاف

مواضيع خدمات الويب الإضافية

هذه المواضيع تكمل الشرائح الـ 15 الأساسية وتمثل أفضل الممارسات والميزات الإضافية

16. التحقق من الصحة (Validation)

ما هو؟ فحوصات من جانب الخادم تضمن أن الطلبات الواردة لها الشكل والقيم المتوقعة قبل المعالجة

أين يقع في الكود:

برمجة التحقق في `backend/src/middleware/validation.js` :

```

export const validation = (schema) => {
  return (req, res, next) => {
    const filter = {...req.body, ...req.params, ...req.query}
    const { error } = schema.validate(filter, {abortEarly: false})
  }
}

```



```

    if (error) {
      throw new AppError(
        error.details.map((d) => d.message),
        400
      )
    }
    next()
  }
}

```

لماذا يهم؟ يمنع البيانات المشوهة، يتجنب عمليات قاعدة البيانات غير الضرورية، ويعيد أخطاء واضحة للعميل (400) في وقت مبكر.

مثال الكود:

```
app.post('/api/v1/auth/signup', validate(signupSchema), signupController)
```

17. الترتيم وتصفية البيانات (Pagination & Filtering)

القوائم Endpoints ما هو؟ أدوات تضيف الترتيم والتصفية والفرز والبحث بالكلمات الرئيسية إلى

أين يقع في الكود؟

- مساعد: [backend/src/utls/apiFeatures.js](#)

لماذا يهم؟ يحسن تجربة المستخدم والأداء بإرجاع فقط الجزء المطلوب من البيانات وتفعيل الاستعلامات المرنة

مثال الكود:

```

const features = new ApiFeatures(Product.find(), req.query)
  .filter().sort().pagination()
const products = await features.mongooseQuery

```

18. متغيرات البيئة والإعدادات

Base URL و PORT ما هو؟ تحميل متغيرات البيئة وتكوين معاملات وقت التشغيل مثل

أين يقع في الكود؟

- البدء: [backend/index.js](#) (`dotenv.config()` وإعداد baseURL/PORT)

يتجنب القيم المدرجة مباشرة، (dev/stage/prod) **لماذا يهم؟** يبقي الأسرار خارج الكود، يسمح بإعدادات مختلفة لكل بيئة

مثال الكود:

```

import dotenv from 'dotenv'
dotenv.config()
const PORT = process.env.PORT || 3000

```

19. عناوين الأمان CORS

(سياسة الأمان، إلخ، CORS) API التي تحمي ال HTTP وعناوين CORS ما هو؟ إعدادات

أين يقع في الكود؟

- CORS الأساسي: [backend/index.js](#) (`app.use(cors())`)

ويخفف من الثغرات الويب الشائعة API لماذا يهم؟ يمنع الأصول غير المأذون بها من استدعاء الـ

مثال الكود:

```
import helmet from 'helmet'
app.use(cors())
app.use(helmet())
```

20. تسجيل الطلبات والتتبع

ما هو؟ تسجيل الطلبات والأخطاء للمراقبة (من استدعى ماذا ومتى)، اختياري مع معرفات التتبع

أين يقع في الكود؟

- سجل الطلبات: [backend/index.js](#) (`morgan('dev')`)

لماذا يهم؟ ضروري للتصحيح وتشخيص مشاكل الإنتاج وتدقيق الطلبات؛ زوجها مع سجلات دائمة

مثال الكود:

```
import morgan from 'morgan'
app.use(morgan('combined'))
```

21. نماذج البيانات والفهرسة

والفهارس التي تحدد البنية وتسرع الاستعلامات على المجموعات الكبيرة Mongoose ما هو؟ مخططات

أين يقع في الكود؟

- النماذج: [backend/database/models/*.model.js](#)

لماذا يهم؟ تصميم المخطط الصحيح والفهارس يجعل الاستعلامات فعالة ويفرض سلامة البيانات

مثال الكود:

```
const schema = new mongoose.Schema({
  title: String,
  price: Number
})
schema.index({ title: 'text' })
```

22. نصائح الأمان

في الإنتاج، إدارة الأسرار، تطهير الإدخال HTTPS، ما هو؟ مجموعة من أفضل الممارسات: تجزئة كلمات المرور

أين يقع في الكود؟

- تجزئة المستخدم: [backend/database/models/user.model.js](#)
- استخدام البيئة: [backend/index.js](#)

لماذا يهم؟ يمنع تسرب بيانات اعتماد المستخدم، يحمي البيانات في النقل، يقلل سطح الهجوم

مثال الكود

```
user.password = await bcrypt.hash(plainPassword, 10)
```

23. التوصيات للإنتاج

API ما هو؟ تحسينات مقترحة: كاش موزع، تسجيل مركزي، اختبارات آلية، مراقبة، تحديد إصدارات الـ

لماذا يهم؟ تجعل الخدمة موثوقة وقابلة للملاحظة وسهلة الاختبار وجاهزة لحركة الإنتاج

مثال الكود

```
import Redis from 'ioredis'
const redis = new Redis(process.env.REDIS_URL)
```

ترتيب العرض الموصى به

1. نظرة عامة على المشروع
2. REST API وأفعال HTTP
3. وسير العملية من العميل إلى الخادم JSON
4. المصادقة والتحقق
5. تكامل قاعدة البيانات
6. رفع الملفات
7. خدمة البريد الإلكتروني
8. Swagger والتوثيق
9. معالجة الأخطاء
10. الكاش والأداء
11. تحديد معدل الطلبات
12. النصائح الختامية

نصائح المقدم

لتبدو طبيعياً، اشرح كل شريحة بهذا الترتيب

1. ما هو؟ - اشرح المفهوم بوضوح
2. أين يقع؟ - أشر إلى الملفات والكود المحدد
3. لماذا يهم؟ - اذكر القيمة والتأثير الحقيقي

الملاحظات المهمة

- له مثال كود حقيقي يعمل Endpoint كل
- جميع مسارات الملفات دقيقة وقابلة للربط
- يمكن نسخ الأكواد مباشرة وتشغيلها
- النظام موحد: كل موضوع يتبع نفس البنية